

TLM: Deep Learning

Chapter 3: Convolutional Neural Network

Outline:

- (1) Problem in dense networks for image dataset
- (2) Introduction to CNN
- (3) Deep convolutional neural networks (DCNN)
- (4) improving the performance with deeper network and data augmentation

Dosen Penyusun:

David Agustriawan, S.Kom., M.S.c., Ph.D.

Ajie Kusuma Wardhana, S.Kom., M.Eng

Problem in dense networks for image dataset

Previously we have discussed dense networks, in which each layer is fully connected to the adjacent layers. In MNIST handwritten classification project, each pixel in the input image has been assigned to a neuron for a total of 784 (28 x 28 pixels) input neurons. **However, this strategy does not leverage the spatial structure and relationship between each image.**

This piece of code is a dense network that transforms the bitmap representing each written digit into a flat vector where the local spatial structure is removed. Removing the spatial structure is a problem because important information is lost:

#X_train is 6000 rows of 28x28 values; we → reshape it to 6000 x 784

RESHAPE = 784

#

X_train = X_train.reshape(60000, RESHAPE)

X_test = X_test.reshape(10000, RESHAPE)

Introduction to CNN

CNN leverage spatial information, and they are therefore very well-suited for classifying images. CNN is based on multiple cortex levels. Each one recognizing more and more structured information. First, it sees single pixels, then from that, we recognize simple geometric forms and then more and more sophisticated elements such as objects, faces, human bodies, animals, and soon. Over a short period, CNN have shown to be disruptive technology, breaking performance records in multiple domains from **text, to video, to speech, going well beyond the initial image processing domain where they were originally conceived**. CNN also known as CNNs, DCNNs, and ConvNets.

Deep convolutional neural networks (DCNN)

DCNN consist of many neural network layers. **Two different type of layers, convolutional and pooling (i.e., subsampling) are typically alternated**. The depth of each filter increases from left to right in the network. The last stage is typically made of one or more fully connected layers.

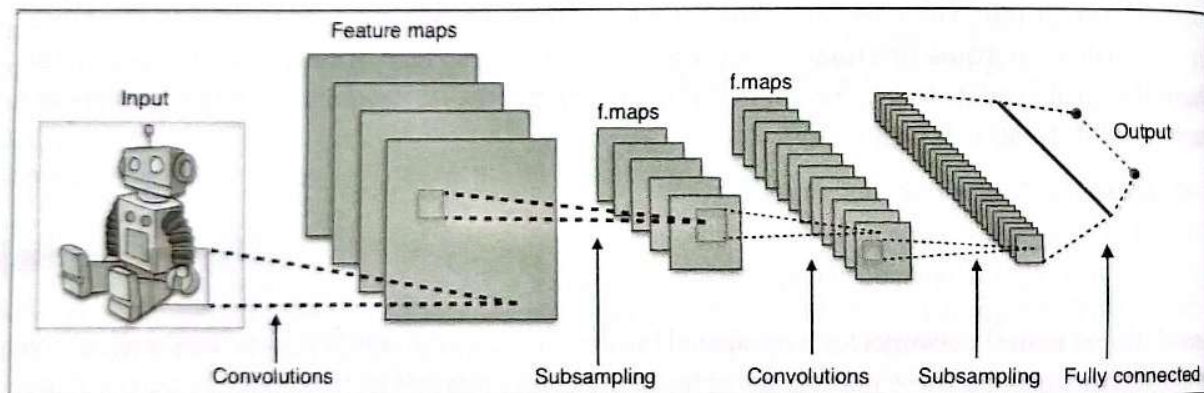


Figure 3.1: An example of a DCNN

There are three key underlying concepts for ConvNets: **Local receptive fields, shared weights, and pooling**:

Local receptive fields

If we want to preserve the spatial information of an image or other form of data, then it is convenient to represent each image with a matrix of pixels. Given this, a simple way to encode the local structure is to connect a submatrix of adjacent input neurons into one single hidden neuron belonging to the next layer. That single hidden neuron represents one local receptive field. Note that this operation is named convolution, and this is where the name for this type of network is derived. Convolution can be as a treatment of a matrix by another matrix, referred to as a kernel.

We can encode more information by having overlapping submatrices. For instance: suppose the size of every single submatrix is 5x5 and those submatrices are used with MNIST images of 28x28 pixels. Then

we will be able to generate 24x24 local receptive fields neurons in the hidden layers. In fact, it is possible to slide the submatrices by only 23 positions before touching the border of the images.

In TensorFlow, the number of pixels along one edge of the kernel, or submatrix, is the kernel size, and the stride length is the number of pixels by which the kernel is moved at each step in the convolution.

Let's define the feature map from one layer to another. We can have multiple feature maps that learn independently from each hidden layer. For example, with 28x28 input neurons for processing MNIST images, and then define k feature maps of size 24x24 neurons each (again with shape of 5x5) in the next hidden layer.

Shared weights and bias

Suppose we want to move away from pixel representation in a raw image, by gaining the ability to detect the same feature independently from the location where it is placed in the input image. A simple approach is to use the same set of weights and biases for all the neuron in the hidden layers. In this way, each layer will learn a set of position-independent latent features derived from the image, bearing in mind that a layer consists of a set of kernels in parallel, and each kernel only learn one feature.

A mathematical example

One simple way to understand convolution is to think about a sliding window function applied to a matrix. Given the input matrix I and the kernel K, we get the convolved output. **The 3x3 kernel K (sometimes called the filter or feature detector)** is multiplied elementwise with the input matrix to get one cell in the output matrix. All others cells are obtained by sliding the window over I.

J	K	Convolved																																											
<table><tr><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td></tr></table>	1	1	1	0	0	0	1	1	1	0	0	0	1	1	1	0	0	1	1	0	0	1	1	0	0	<table><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr></table>	1	0	1	0	1	0	1	0	1	<table><tr><td>4</td><td>3</td><td>4</td></tr><tr><td>2</td><td>4</td><td>3</td></tr><tr><td>2</td><td>3</td><td>4</td></tr></table>	4	3	4	2	4	3	2	3	4
1	1	1	0	0																																									
0	1	1	1	0																																									
0	0	1	1	1																																									
0	0	1	1	0																																									
0	1	1	0	0																																									
1	0	1																																											
0	1	0																																											
1	0	1																																											
4	3	4																																											
2	4	3																																											
2	3	4																																											

In this example, we decided to stop the sliding window as soon as we touch the borders of I (so the output is 3x3). Alternatively, we could have chosen to pad the input with zero (so that the output would have been 5x5). This decision relates to the padding choice adopted. Note that kernel depth is equal to input depth (channel).

Another choice is about how far along we slide our sliding windows with each step. This is called the stride and it can be one or more. A larger stride generates fewer applications of the kernel and a smaller output size, while a smaller stride generates more output and retains more information. **The size of filter, the stride, and the type of padding are hyperparameters that can be fine-tuned during the training of the network.**

ConvNets in TensorFlow

In TensorFlow, if we want to add a convolutional layer with 32 parallel features and a filter size of 3x3,

We write:

```
import tensorflow as tf  
  
from tensorflow.keras import datasets, layers, models  
  
model = models.Sequential()  
  
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
```

This means that we are applying a 3x3 convolution on 28x28 images with 1 input channel (or input filters) resulting in 32 output channels (or output filters).

An example of convolution is provided in figure 3.2 below:

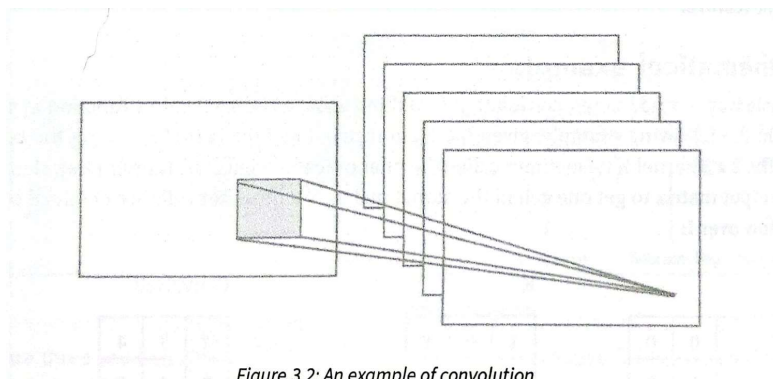


Figure.3.2: An example of convolution

Pooling Layers

Suppose we want to summarize the output of a feature map. We can use the spatial contiguity of the output produced from a single feature map and aggregate the values of a sub-matrix into one single output value synthetically describing the “meaning” associated with the physical region.

Max pooling

One easy and common choice is the so called max pooling operator, which simply outputs the maximum activation as observed in the region. In Keras, if we want to define a max pooling layer of size 2x2, we write:

```
model.add(layers.MaxPooling2D((2, 2)))
```

An example of the max-pooling operation is given in figure 3.3:

1	0	3	6
2	4	5	2
2	6	2	0
3	4	1	7

4	6
6	7

Figure 3.3: An example of max pooling

Average pooling

Another choice is the average pooling, which simply aggregates a region into the average values of the activations observed in that region. Keras implements a large number of pooling layers, and a complete list is available online (see <https://keras.io/layers/pooling/>). In short, all the pooling operations are nothing more than a summary operation on a given region.

ConvNets summary

ConvNets/CNNs apply convolution and pooling operations in **one dimension for audio and text data along the time dimension**, in **two dimensions for images along the (height x width) dimensions**, and in **three dimensions for videos along the (height x width x time) dimensions**. For images, sliding the filter over an input volume produces a map that provides the responses of the filter for each spatial position. In other words, a CNN has multiple filters stacked together that learn to recognize specific visual features independently from the location in the image itself. Those visual features are simple in the initial layers of the network and become more and more sophisticated deeper in the network. Training of a CNN requires the identification of the right values for each filter so that an input, when passed through multiple layers, activates certain neurons of the last layer so that it will predict the correct values.

An example of DCNN: leNet

Yann LeCun the winner of the Turing award proposed a family of ConvNets named LeNet, trained for recognizing MNIST handwritten characters with robustness to simple geometric transformations and distortion. The core idea of LeNet is to have lower layers alternating convolution operations with max-pooling operations. The convolution operations are based on carefully chosen local receptive fields with

shared weights for multiple feature maps. Then, higher levels are fully connected based on traditional MLP with hidden layers and softmax as the output layer.

LeNet code in TF

Using a convolution 2D module:

```
Layers.Convolution2D(20, (5, 5), activation='relu', input_shape=input_shape)
```

Where the first parameter is the number of output filters in the convolution and the next tuple is the extension of each filter. An interesting additional parameter is padding. There are two options: padding = 'valid' means the convolution is only computed where the input and the filter fully overlap and therefore the output is smaller than the input, while padding = 'same' means that we have an output that is the same size as the input, for which the area around the input is padded with zeros.

In addition, we use MaxPooling2D module:

```
Layers.MaxPooling2D(pool_size=(2, 2), strides=(2, 2))
```

Where pool_size = (2,2) is a tuple of 2 integers representing the factors by which the image is vertically and horizontally downsampled. So (2, 2) will halve the image in each dimension, and strides = (2, 2) is the stride use for processing.

Let's review the code. First, import a number of modules:

```
import tensorflow as tf

from tensorflow.keras import datasets, layers, models, optimizers

#network and training

EPOCHS = 5

BATCH_SIZE = 128

VERBOSE = 1

OPTIMIZER = tf.keras.optimizers.Adam()

VALIDATION_SPLIT=0.90

  
IMG_ROWS, IMG_COLS = 28, 28 # input image dimensions

INPUT_SHAPE = (IMG_ROWS, IMG_COLS, 1)

NB_CLASSES = 10 # number of outputs = number of digits
```

Then define the LeNet network:

```
#define the convent
```

```
def build(input_shape, classes):
```

```
    model = models.Sequential()
```

We have a first convolutional stage with ReLu activations followed by max pooling. Our network will learn 20 convolutional filters, each of which has size of 5x5. The output dimension is the same as the input_shape, so it will be 28x28. Since Convolutional2D is the first stage of the pipeline, so it is also required to define its input_shape. The max pooling operation implements a sliding window which slides over the layer and takes the maximum of each region with a step of two pixels both vertically and horizontally.

```
#CONV -> RELU - > POOL
```

```
model.add (layers.Convolution2D(20, (5, 5), activation='relu',
```

```
    input_shape=input_shape))
```

```
model.add(layers.MaxPooling2D(pool_size = (2, 2), strides = (2, 2)))
```

Then, there is a second convolutional stage with ReLu activations, followed again by a max pooling layer. In this case, there are increasing number of convolutional filters learned to 50 from previously 20. Increasing the number of filters in deep layers is a common technique in deep learning:

```
#CONV -> RELU - > POOL
```

```
model.add (layers.Convolution2D(50, (5, 5), activation='relu'))
```

```
model.add(layers.MaxPooling2D(pool_size = (2, 2), strides = (2, 2)))
```

Then we have a pretty standard flattening and a dense network of 500 neurons, followed by a softmax classifiers with 10 classes:

```
#Flatten -> RELU Layers
```

```
model.add(layers.Flatten())
```

```
model.add(layers.Dense(500, activation='relu'))
```

```
# a softmax classifier
```

```
model.add(layers.Dense(classes, activation='softmax'))
```

```
return model
```

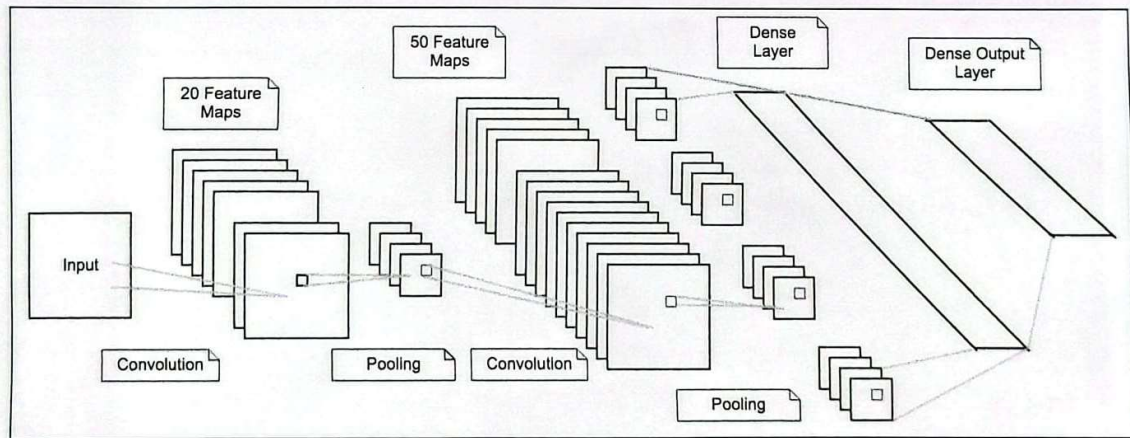


Figure 3.4: Visualization of LeNet

The rest of the code, including the code for printing the loss:

#data: shuffled and split between train and test sets

```
(X_train, y_train), (X_test, y_test) = datasets.mnist.load_data()
```

#reshape

```
X_train = X_train.reshape((60000, 28, 28, 1))
```

```
X_test = X_test.reshape((10000, 28, 28, 1))
```

#normalize

```
X_train, X_test = X_train / 255.0, X_test / 255.0
```

#cast

```
X_train = X_train.astype('float32')
```

```
X_test = X_test.astype('float32')
```

#convert class vectors to binary class matrices

```
y_train = tf.keras.utils.to_categorical(y_train, NB_CLASSES)
```

```
y_test = tf.keras.utils.to_categorical(y_test, NB_CLASSES)
```

#initialize the optimizer and model

```
model = LeNet.build(input_shape=INPUT_SHAPE, classes = NB_CLASSES)
```

```
model.compile(loss="categorical_crossentropy", optimizer = OPTIMIZER, metrics = ["accuracy"])  
model.summary()
```

```
#use TensorBoard
```

```
Callbacks = [  
    #write tensorboard logs to './logs' directory  
    tf.keras.callbacks.TensorBoard(log_dir='./logs')  
]
```

```
#fit
```

```
history = model.fit(X_train, y_train,  
                    batch_size=BATCH_SIZE, epochs=EPOCHS,  
                    verbose=verbose, validation_split=VALIDATION_SPLIT,  
                    callbacks=callbacks)  
score = model.evaluate(X_test, y_test, verbose=VERBOSE)  
print ("\nTest score:", score[0])  
print ("Test accuracy:", score[1])
```

as shown in figure 3.5, the time had significant increase, and each iteration in our deep net now takes ~28 seconds against ~1-2 seconds for the network defined during teaching learning in session 1. However, the accuracy reached a new peak at 99.991% on training, 99.91% on validation, and 99.15% on test.

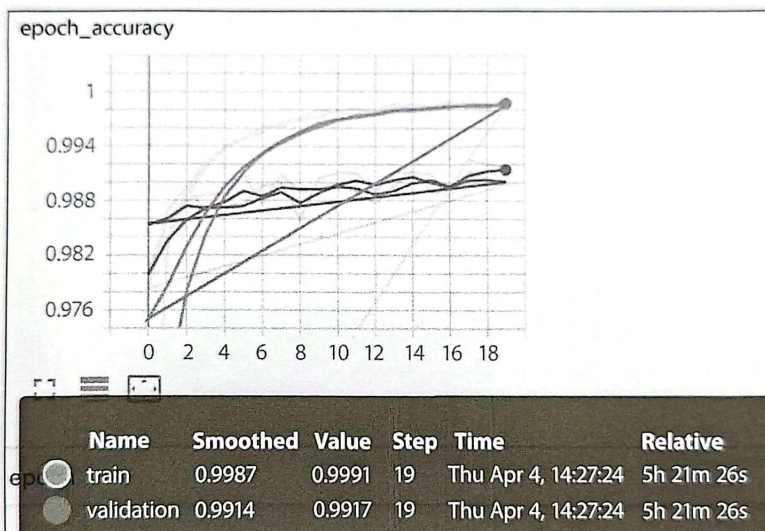


Figure 3.5: LeNet accuracy

Let's see the execution of a full run for 20 epochs:

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 24, 24, 20)	520
max_pooling2d_2 (MaxPooling 2D)	(None, 12, 12, 20)	0
conv2d_3 (Conv2D)	(None, 8, 8, 50)	25050
max_pooling2d_3 (MaxPooling 2D)	(None, 4, 4, 50)	0
flatten (Flatten)	(None, 800)	0
dense (Dense)	(None, 500)	400500
dense_1 (Dense)	(None, 10)	5010

Total params: 431,080
Trainable params: 431,080
Non-trainable params: 0

Train on 48000 samples, validate on 12000 samples

Epoch 1/20

[2019-04-04 14:18:28.546158: I tensorflow/core/profiler/lib/profiler_session.cc:164] Profile Session started.

48000/48000 [=====] - 28s 594us/sample - loss: 0.2035
- accuracy: 0.9398 - val_loss: 0.0739 - val_accuracy: 0.9783

Epoch 2/20

48000/48000 [=====] - 26s 534us/sample - loss: 0.0520
- accuracy: 0.9839 - val_loss: 0.0435 - val_accuracy: 0.9868

Epoch 3/20

48000/48000 [=====] - 27s 564us/sample - loss: 0.0343
- accuracy: 0.9893 - val_loss: 0.0365 - val_accuracy: 0.9895

Epoch 4/20

48000/48000 [=====] - 27s 562us/sample - loss: 0.0248
- accuracy: 0.9921 - val_loss: 0.0452 - val_accuracy: 0.9868

Epoch 5/20

48000/48000 [=====] - 27s 562us/sample - loss: 0.0195
- accuracy: 0.9939 - val_loss: 0.0428 - val_accuracy: 0.9873

Epoch 6/20

48000/48000 [=====] - 28s 548us/sample - loss: 0.0585
- accuracy: 0.9820 - val_loss: 0.1038 - val_accuracy: 0.9685

Epoch 7/20

48000/48000 [=====] - 26s 537us/sample - loss: 0.0134
- accuracy: 0.9955 - val_loss: 0.0388 - val_accuracy: 0.9896

Epoch 8/20

48000/48000 [=====] - 29s 589us/sample - loss: 0.0097
- accuracy: 0.9966 - val_loss: 0.0347 - val_accuracy: 0.9899

Epoch 9/20

48000/48000 [=====] - 29s 607us/sample - loss: 0.0091
- accuracy: 0.9971 - val_loss: 0.0515 - val_accuracy: 0.9859

Epoch 10/20

48000/48000 [=====] - 27s 565us/sample - loss: 0.0062
- accuracy: 0.9980 - val_loss: 0.0376 - val_accuracy: 0.9904

Epoch 11/20

48000/48000 [=====] - 30s 627us/sample - loss: 0.0068

```

- accuracy: 0.9976 - val_loss: 0.0366 - val_accuracy: 0.9911
Epoch 12/20
48000/48000 [=====] - 24s 505us/sample - loss: 0.0079
- accuracy: 0.9975 - val_loss: 0.0389 - val_accuracy: 0.9910
Epoch 13/20
48000/48000 [=====] - 28s 584us/sample - loss: 0.0057
- accuracy: 0.9978 - val_loss: 0.0531 - val_accuracy: 0.9890
Epoch 14/20
48000/48000 [=====] - 28s 580us/sample - loss: 0.0045
- accuracy: 0.9984 - val_loss: 0.0409 - val_accuracy: 0.9911
Epoch 15/20
48000/48000 [=====] - 26s 537us/sample - loss: 0.0039
- accuracy: 0.9986 - val_loss: 0.0436 - val_accuracy: 0.9911
Epoch 16/20
48000/48000 [=====] - 25s 513us/sample - loss: 0.0059
- accuracy: 0.9983 - val_loss: 0.0480 - val_accuracy: 0.9890
Epoch 17/20
48000/48000 [=====] - 24s 499us/sample - loss: 0.0042
- accuracy: 0.9988 - val_loss: 0.0535 - val_accuracy: 0.9888
Epoch 18/20
48000/48000 [=====] - 24s 505us/sample - loss: 0.0042
- accuracy: 0.9986 - val_loss: 0.0349 - val_accuracy: 0.9926
Epoch 19/20
48000/48000 [=====] - 29s 599us/sample - loss: 0.0052
- accuracy: 0.9984 - val_loss: 0.0377 - val_accuracy: 0.9920
Epoch 20/20
48000/48000 [=====] - 25s 524us/sample - loss: 0.0028
- accuracy: 0.9991 - val_loss: 0.0477 - val_accuracy: 0.9917
10000/10000 [=====] - 2s 248us/sample - loss: 0.0383 -
accuracy: 0.9915

Test score: 0.03832608199457617
Test accuracy: 0.9915

```

Let's plot the model accuracy and the model loss, and we understand that we can train in only 10 iterations to achieve a similar accuracy of 99.1%:

```

Train on 48000 samples, validate on 12000 samples
Epoch 1/10
[2019-04-04 15:57:17.848186: I tensorflow/core/profiler/lib/profiler_session.
cc:164] Profile Session started.
48000/48000 [=====] - 26s 544us/sample - loss: 0.2134
- accuracy: 0.9361 - val_loss: 0.0688 - val_accuracy: 0.9783
Epoch 2/10

```

```

48000/48000 [=====] - 30s 631us/sample - loss: 0.0550
- accuracy: 0.9831 - val_loss: 0.0533 - val_accuracy: 0.9843
Epoch 3/10
48000/48000 [=====] - 30s 621us/sample - loss: 0.0353
- accuracy: 0.9884 - val_loss: 0.0410 - val_accuracy: 0.9874
Epoch 4/10
48000/48000 [=====] - 37s 767us/sample - loss: 0.0276
- accuracy: 0.9910 - val_loss: 0.0381 - val_accuracy: 0.9887
Epoch 5/10
48000/48000 [=====] - 24s 509us/sample - loss: 0.0200
- accuracy: 0.9932 - val_loss: 0.0406 - val_accuracy: 0.9881
Epoch 6/10
48000/48000 [=====] - 31s 641us/sample - loss: 0.0161
- accuracy: 0.9950 - val_loss: 0.0423 - val_accuracy: 0.9881
Epoch 7/10
48000/48000 [=====] - 29s 613us/sample - loss: 0.0129
- accuracy: 0.9955 - val_loss: 0.0396 - val_accuracy: 0.9894
Epoch 8/10
48000/48000 [=====] - 27s 554us/sample - loss: 0.0107
- accuracy: 0.9965 - val_loss: 0.0454 - val_accuracy: 0.9871
Epoch 9/10
48000/48000 [=====] - 24s 510us/sample - loss: 0.0082
- accuracy: 0.9973 - val_loss: 0.0388 - val_accuracy: 0.9902
Epoch 10/10
48000/48000 [=====] - 26s 542us/sample - loss: 0.0083
- accuracy: 0.9970 - val_loss: 0.0440 - val_accuracy: 0.99892
10000/10000 [=====] - 2s 196us/sample - loss: 0.0327 -
accuracy: 0.9910

Test score: 0.03265062951518773
Test accuracy: 0.991

```

Let us see some of the MNIST images just to understand how good the number 99.1% is!

For instance, there are many ways in which humans write a 9, one of them being in figure 3.6. the same goes for 3, 7, 4, and 5, and number 1 in this figure is so difficult to recognize that even a human would likely have trouble:

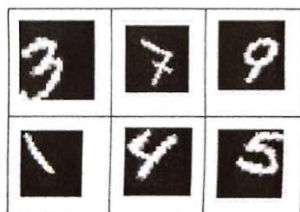


Figure 3.6: An example of MNIST handwritten characters

This is the summarize from session 1 results of various model that have been built with worst score accuracy **90.71%** meaning that **about 9 handwritten characters out of 100 are not correctly recognized**. Then, gain 8% with deep learning architecture, reaching an accuracy of **99.2%** which means that **less than one handwritten character out of hundred is incorrectly recognized** as shown in figure 3.7.

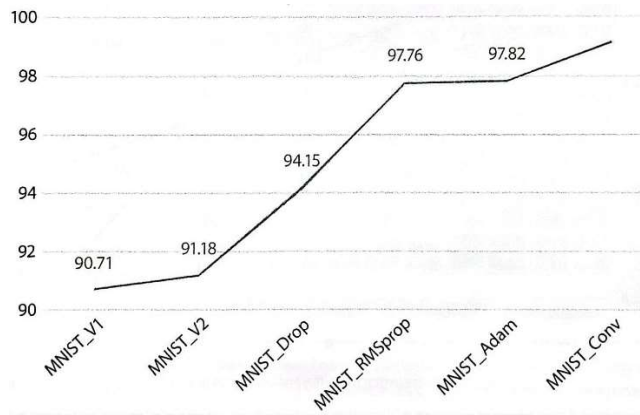


Figure 3.7: Accuracy for different models and optimizers

Understanding the power of deep learning

Another test we can run for a better understanding of the power of deep learning and ConvNets is to reduce the size of the training set and observe the resulting decay in performance. One way to do this is to split the training set of 50,000 examples into two different sets:

- The proper training set used for training the model will progressively reduce in size: 5900, 3000, 1800, 600, and 300 examples.
- The validation set used to estimate how well our model has been trained will consist of the remaining examples. Our test set is always fixed, and it is consisting of 10,000 examples.

As the results shown below, our deep network always outperforms the simple network when there is more data available. With 5 900 training examples, the deep learning net had an accuracy of 97.23% against an accuracy of 94% for the simple net. In general, deep networks require more training data available to fully express their power as depicted in figure 3.8:

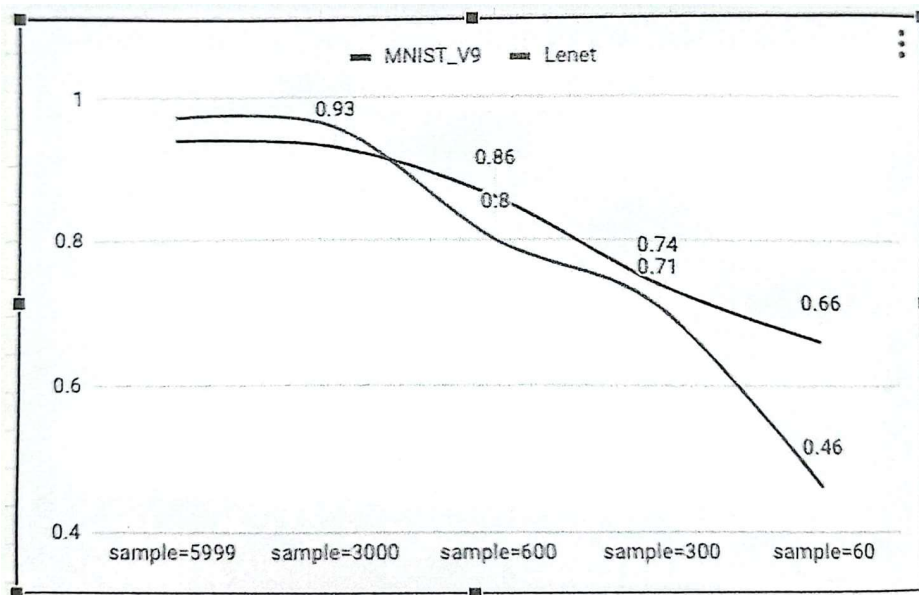


Figure 3.8: Accuracy for different amounts of data

Recognizing CIFAR-10 images with deep learning

The CIFAR-10 dataset contains 60,000 color images of 32x32 pixels in three channels, divided into 10 classes. Each class contains 6,000 images. The training set contains 50,000 images, while the test set provides 10,000 images.

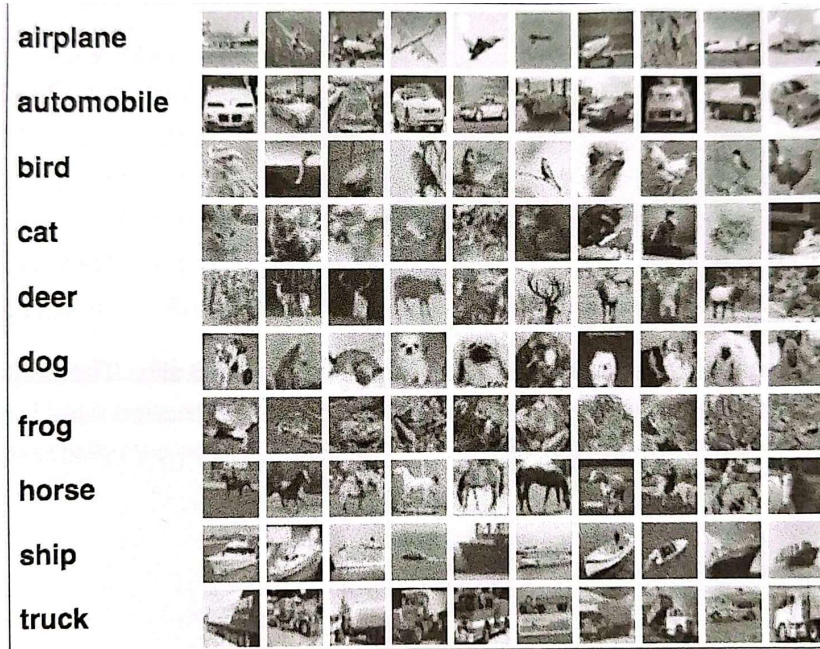


Figure 3.9: An example of CIFAR-10 images

The goal is to recognize previously unseen images and assign them to one of the ten classes. Let's define suitable deep net.

First of all, import a number of useful modules and define a few constants and load the dataset:

```
import tensorflow as tf

from tensorflow.keras import datasets, layers, models, optimizers

# CIFAR_10 is a set of 60K images 32x32 pixels on 3 channels

IMG_CHANNELS = 3

IMG_ROW = 32

IMG_COLS = 32

#constant

BATCH_SIZE = 128

EPOCHS = 20

CLASSES = 10
```

```
VERBOSE = 1
```

```
VALIDATION_SPLIT = 0.2
```

```
OPTIM = tf.keras.optimizer.RMSprop()
```

Our net will learn 32 convolutional filters, each of which with 3x3 size. The output dimension is the same as the input shape, so it will be 32x32 and the activation function used is a ReLu function which is a simple way to introducing non-linearity. After that, we have a MaxPooling operation with a pool size of 2x2 and dropout at 25%:

```
#define the convent
```

```
def build(input_shape, classes):
```

```
    model = models.Sequential()
```

```
    model.add(layers.Convolution2D(32, (3, 3), activation='relu',  
                                input_shape=input_shape))
```

```
    model.add(layers.MaxPooling2D(pool_size = (2, 2)))
```

```
    model.add(layers.Dropout(0.25))
```

The next stage in the deep pipeline is a dense network with 512 units and ReLu activation followed by dropout 50% and by a softmax layer with 10 classes as output, one for each category:

```
    model.add(layers.Flatten())
```

```
    model.add(layers.Dense(512, activation='relu'))
```

```
    model.add(layers.Dropout(0.5))
```

```
    model.add(layers.Dense(classes, activation = 'softmax'))
```

```
    return model
```

After defining the network, we can train the model. In this case, we split the data and compute a validation set in addition to the training and testing sets. The training is used to build our models, the validation is used to select the best performing approach, while the test set is used to check the performance of our best models on fresh unseen data.

```
#use TensorBoard
```

```
Callbacks = [
```

```
    #write tensorboard logs to './logs' directory
```

```
    tf.keras.callbacks.TensorBoard(log_dir='./logs')
```

```
]
```

```
#train
```

```
model.compile(loss='categorical_crossentropy', optimizer=OPTIM, metrics = ['accuracy'])
```

```
#fit
model.fit(X_train, y_train, batch_size=BATCH_SIZE, epochs=EPOCHS,
          verbose=verbose, validation_split=VALIDATION_SPLIT,
          callbacks=callbacks)

score = model.evaluate(X_test, y_test, batch_size = BATCH_SIZE, verbose=VERBOSE)

print ("\nTest score:", score[0])
print('Test accuracy:', score[1])
```

The result reaches a test accuracy of 66.8% with 20 iterations. We also print the accuracy and loss plot and dump the network with `model.summary()`:

```

Epoch 17/20
40000/40000 [=====] - 112s 3ms/sample - loss: 0.6282 -
accuracy: 0.7841 - val_loss: 1.0296 - val_accuracy: 0.6734
Epoch 18/20
40000/40000 [=====] - 76s 2ms/sample - loss: 0.6140 -
accuracy: 0.7879 - val_loss: 1.0789 - val_accuracy: 0.6489
Epoch 19/20
40000/40000 [=====] - 74s 2ms/sample - loss: 0.5931 -
accuracy: 0.7958 - val_loss: 1.0461 - val_accuracy: 0.6811
Epoch 20/20
40000/40000 [=====] - 71s 2ms/sample - loss: 0.5724 -
accuracy: 0.8042 - val_loss: 0.1.0527 - val_accuracy: 0.6773
10000/10000 [=====] - 5s 472us/sample - loss: 1.0423 -
accuracy: 0.6686

Test score: 1.0423416819572449
Test accuracy: 0.6686

```

Figure 3.10 shows the accuracy and loss plot:

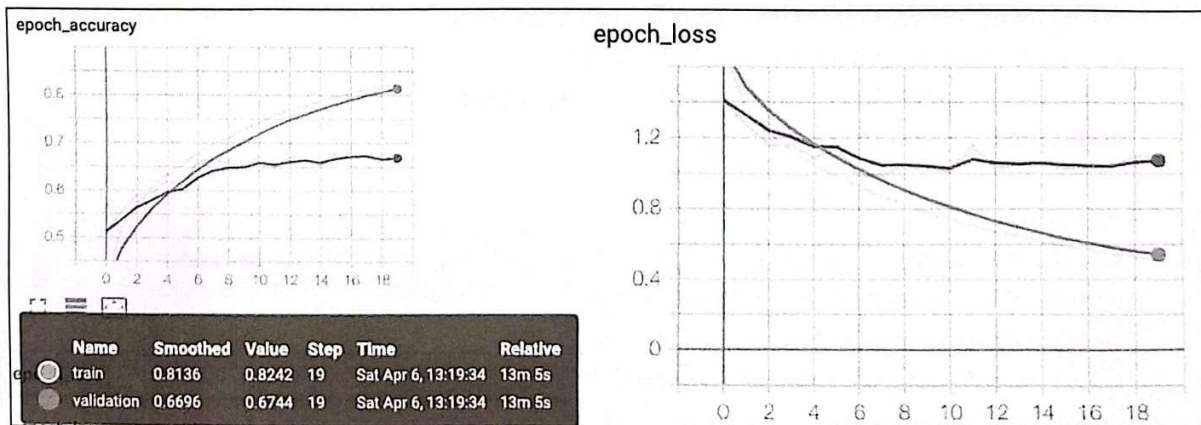


Figure 3.10: Accuracy and loss for the defined network

Improving the CIFAR-10 performance with deeper network

We can add multiple convolutional operations. In the following example, we have a sequence of modules:

1st module: (CONV+CONV+MaxPool+DropOut)

2nd module: (CONV+CONV+MaxPool+DropOut)

3rd module: (CONV+CONV+MaxPool+DropOut)

These are followed by a standard dense output layer. All the activation functions used are ReLu functions. There is a new layer `batchNormalization()` used to introduce a form of regularization between modules:

```
def build_model():
```

```
    model = models.Sequential()
```

```
    #1st block
```

```
    model.add(layers.Convolution2D(32, (3, 3), padding = 'same', input_shape = x_train.shape[1:],  
activation='relu'))
```

```
    model.add(layers.BatchNormalization())
```

```
    model.add(layers.Convolution2D(32, (3, 3), padding = 'same', activation='relu'))
```

```
    model.add(layers.BatchNormalization())
```

```
    model.add(layers.MaxPooling2D(pool_size = (2, 2)))
```

```
    model.add(layers.Dropout(0.2))
```

```
    #2nd block
```

```
    model.add(layers.Convolution2D(64, (3, 3), padding = 'same', activation='relu'))
```

```
    model.add(layers.BatchNormalization())
```

```
    model.add(layers.Convolution2D(64, (3, 3), padding = 'same', activation='relu'))
```

```
    model.add(layers.BatchNormalization())
```

```
    model.add(layers.MaxPooling2D(pool_size = (2, 2)))
```

```
    model.add(layers.Dropout(0.3))
```

```
    #3rd block
```

```
    model.add(layers.Convolution2D(128, (3, 3), padding = 'same', activation='relu'))
```

```
    model.add(layers.BatchNormalization())
```

```
    model.add(layers.Convolution2D(64, (3, 3), padding = 'same', activation='relu'))
```

```
    model.add(layers.BatchNormalization())
```

```
    model.add(layers.MaxPooling2D(pool_size = (2, 2)))
```

```
    model.add(layers.Dropout(0.4))
```

```

#dense
model.add(layers.Flatten())

model.add(layers.Dense(NUM_CLASSES, activation='softmax'))

return model

model.summary()

```

From those network above the accuracy is increase to 82%. Here is the remaining code required:

```

import tensorflow as tf

from tensorflow.keras import datasets, layers, models, regularizer, optimizers

from tensorflow.keras.preprocessing.image import ImageDataGenerator

import numpy as np

```

```

EPOCHS = 50

```

```

NUM_CLASSES = 10

```

```

def load_data():

    (x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()

    x_train = x_train.astype('float32')

    x_test = x_test.astype('float32')


    #normalize

    mean = np.mean(x_train, axis=(0,1,2,3))

    std = np.std(x_train, axis=(0,1,2,3))

    x_train = (x_train - mean) / (std+1e-7)

    x_test = (x_test - mean) / (std+1e-7)


    y_train = tf.keras.utils.to_categorical(y_train, NUM_CLASSES)

    y_test = tf.keras.utils.to_categorical(y_test, NUM_CLASSES)


    return x_train, y_train, x_test, y_test

```

Then we need to have a part to train the network:

```
(x_train, y_train, x_test, y_test) load_data()

Model = build_model()

model.compile(loss='categorical_crossentropy', optimizer=RMSprop, metrics = ['accuracy'])

#train

Batch_size = 64

model.fit(x_train, y_train, batch_size=batch_size, epochs=EPOCHS,
          validation_data=(x_test, y_test))

score = model.evaluate(x_test, y_test, batch_size=batch_size)

print ("\nTest score:", score[0])

print ("Test accuracy:", score[1])
```

as the result, it shows improvement of 15.14% with respect to the previous simpler deeper network.

Improving the CIFAR-10 performance with data augmentation

Another way to improve the accuracy is to generate more images for our training. We can augment the dataset on the standard CIFAR training set with multiple types of transformation, including rotation, rescaling, horizontal or vertical flip, zooming, channel shift, and many more.

The code:

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

#image augmentation

datagen = ImageDataGenerator(
    rotation_range=30,
    width_shift_range=0.2,
    height_shift_range=0.2,
    horizontal_flip = True,
)

datagen.fit(x_train)
```

we can apply that augmentation code directly for training. Using the same ConvNet defined before, we simply generate more augmented images, and then we train. For efficiency, the generator runs in parallel to the model. This allows an image augmentation on a CPU while training in parallel on GPU:

```
#train
```

```
Batch_size = 64
```

```
model.fit_generator(datagen.flow(x_train, y_train, batch_size=batch_size), epochs=EPOCHS, verbose = 1,  
validation_data=(x_test, y_test))
```

```
#save to disk
```

```
model_json = model.to_json()
```

```
with open('model.json', 'w') as json_file:
```

```
    json_file.write(model_json)
```

```
model.save_weights('model.h5')
```

```
#test
```

```
score = model.evaluate(x_test, y_test, batch-size = 128, verbose=1)
```

```
print ("\nTest result: %.3f loss: %.3f" % (score[1]*100, score[0]))
```

each iteration now more expensive because we have more training data. Therefore, let's run for 50 iterations only. It shows the accuracy of 85.91%.

```

Epoch 46/50
50000/50000 [=====] - 36s 722us/sample - loss: 0.2440
- accuracy: 0.9183 - val_loss: 0.4918 - val_accuracy: 0.8546
Epoch 47/50
50000/50000 [=====] - 34s 685us/sample - loss: 0.2338
- accuracy: 0.9208 - val_loss: 0.4884 - val_accuracy: 0.8574
Epoch 48/50
50000/50000 [=====] - 32s 643us/sample - loss: 0.2383
- accuracy: 0.9189 - val_loss: 0.5106 - val_accuracy: 0.8556
Epoch 49/50
50000/50000 [=====] - 37s 734us/sample - loss: 0.2285
- accuracy: 0.9212 - val_loss: 0.5017 - val_accuracy: 0.8581
Epoch 49/50
50000/50000 [=====] - 36s 712us/sample - loss: 0.2263
- accuracy: 0.9228 - val_loss: 0.4911 - val_accuracy: 0.8591
10000/10000 [=====] - 2s 160us/sample - loss: 0.4911 -
accuracy: 0.8591

Test score: 0.4911323667049408
Test accuracy: 0.8591

```

The results obtained during our experiments are summarized in the following figure:

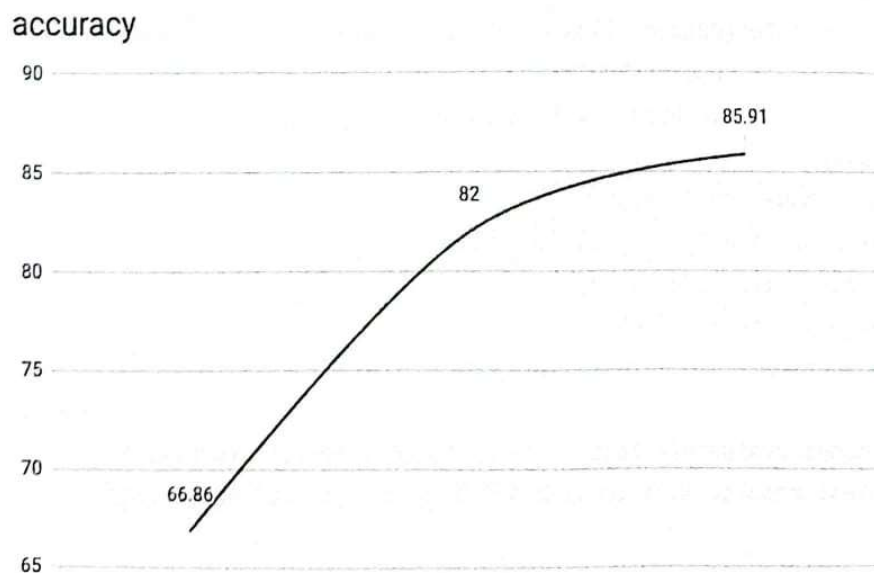


Figure 3.12: Accuracy on CIFAR-10 with different networks. On the x-axis, we have the increasing number of iterations